

RELATÓRIO EXPERIMENTO 8

Nome: Renato Gabriel Lima da Silva

Matrícula: 251027871

INTRODUÇÃO

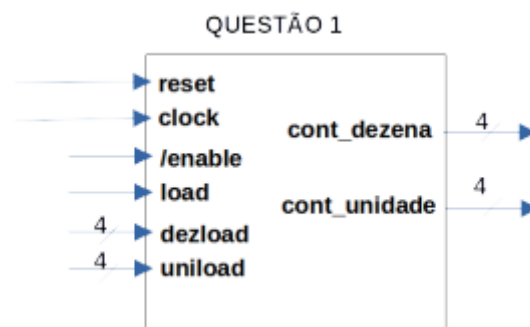
Neste experimento, vamos projetar e implementar, usando a linguagem VHDL, um sistema de controle de semáforos, implementar contadores em VHDL, e fixar o uso de técnicas de projeto modular em VHDL, desenvolvendo grandes sistemas construídos utilizando outros sistemas menores, interligados entre si. em e simular no ModelSim. Vamos abordar seus conceitos e suas funcionalidades.

QUESTÃO 1

TEORIA

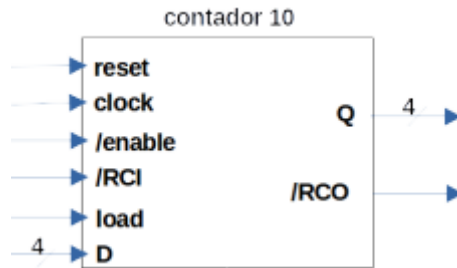
Contador Módulo 100

Na primeira questão vamos implementar um contador módulo 100 com saída BCD, com entradas de 'reset' e 'load' síncronas, clock de 1 Hz e entrada de 'enable' (ativa em nível baixo). A caixa preta e o diagrama de blocos da arquitetura interna desse sistema são apresentados na Figura 1.



- **Figura 1:** Caixa preta do sistema a ser implementado

Para implementarmos, é necessário cascatear dois outros contadores de módulo 10 (figura 2). Um contador BCD módulo 10 é um contador de 4 bits usual, com o diferencial de retornar ao zero quando atinge nove.



- **Figura 2:** Caixa preta do componente contador 10

Um contador BCD módulo 100 é um contador similar ao BCD módulo 10, apenas agora atuando em dois dígitos ao invés de apenas um. Ou seja, ele possui uma saída para a dezena atual, e outra para a unidade atual. Por requisito do experimento, serão necessárias as mesmas entradas e saídas do contador módulo 10, com algumas alterações. Para os valores de carregamento, o load permanece igual, mas agora o contador aceita uma entrada de carregamento para as unidades, uniload, e outra para as dezenas dezload ambas de 4 bits. Similarmente, não há apenas uma saída, mas duas, cont_unidade para as unidades e cont_dezena para as dezenas. Além disso, o experimento requisita que o contador módulo 100 seja construído por cascadeamento, ou seja, deveria ser usado dois contadores módulo 10 para simular cada dígito do contador. No ponto de vista lógico, a saída RCO do contador das unidades deverá ser usada para indicar quando que o contador das dezenas deve incrementar seu valor, enquanto a saída RCO das dezenas servirá para indicar que o contador atualmente se encontra no estado 99.

CÓDIGO

Todas as entradas e saídas serão de apenas um bit, como no contador módulo 10, com exceção das entradas D_uni e D_dez, e as saídas Q_uni e Q_dez, todas as quatro vetores de 4 bits.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity CONTADORBCD100 is
5      port (
6          clock : in  std_logic;
7          reset  : in  std_logic;
8          enable : in  std_logic;
9          rci    : in  std_logic;
10         load   : in  std_logic;
11         d_uni  : in  std_logic_vector(3 downto 0);
12         d_dez  : in  std_logic_vector(3 downto 0);
13         Q_uni  : out std_logic_vector(3 downto 0);
14         Q_dez  : out std_logic_vector(3 downto 0);
15         RCO    : out std_logic
16     );
17 end CONTADORBCD100;
18
19 architecture CONTADORBCD100_ARCH of CONTADORBCD100 is
20
21     -- Declaração do seu componente fornecido
22     component CONTADORBCD10 is
23         port (
24             clock: in std_logic;
25             reset: in std_logic;
26             enable: in std_logic;
27             rci: in std_logic;
28             load: in std_logic;
29             d: in std_logic_vector(3 downto 0);
30             Q: out std_logic_vector(3 downto 0);
31             RCO: out std_logic
32         );
33     end component;
34
35     -- Sinal interno para passar o "vai um" (Ripple Carry) das unidades para as dezenas
36     signal uRCO : std_logic;
37     signal dRCO : std_logic;
38
39 begin
40
41     -- Instância 1: Controla as UNIDADES
42     -- Recebe o enable e o rci externos do sistema.
43     INT1 : CONTADORBCD10
44         port map(
45             clock => clock,
46             reset => reset,
47             enable => enable,

```

```

48         rci    => rci,
49         load   => load,
50         d      => d_uni,
51         Q      => Q_uni,
52         RCO    => uRCO
53     );
54
55     -- Instância 2: Controla as DEZENAS
56     -- Só incrementa se o enable geral estiver ativo ('0') E se a unidade chegou em 9 (uRCO = '0').
57     INT2 : CONTADORBCD10
58         port map(
59             clock => clock,
60             reset => reset,
61             enable => enable,
62             rci   => uRCO, -- O RCO das unidades ativa o rci das dezenas
63             load  => load,
64             d     => d_dez,
65             Q     => Q_dez,
66             RCO   => dRCO
67         );
68
69     -- O RCO do sistema de 100 bits será ativo em '0' quando ambos chegarem em 99
70     RCO <= uRCO or dRCO;
71
72     end CONTADORBCD100_ARCH;

```

Listagem 1 : Código do contador módulo 100 BCD.

O código da Listagem 1 implementa as bibliotecas necessárias (linha 1 e 2), e as portas anteriormente citadas (linhas 6 a 15).

Já para a arquitetura do contador 100, o processo é bem simples. Basta, primeiramente, importar o contador 10 como um componente, aqui nas linhas 22 a 33 da Listagem 1. Em seguida, define-se dois sinais de bit simples, uRCO e dRCO, na linha 36 e 37, para conectar a saída RCO de ambos contadores, unidade e dezena. Finalmente, as conexões são realizadas nas linhas 43 a 67, com INT1 representando o contador das unidades e INT2 representando o contador das dezenas. As entradas clock, reset, enable e load são conectadas identicamente em ambos contadores. A entrada rci do contador 100 é inserida no contador das unidades, e usa-se o sinal uRCO para receber a saída das unidades e conectá-la com a entrada rci do valor das dezenas. Já para os valores de d e Q, usa-se d_uni e Q_uni para as unidades, e d_dez e Q_dez para as dezenas. Por fim, realiza-se a lógica da saída de RCO para o contador 100 na linha 70 da Listagem 1.

Dessa forma, o contador BCD módulo 100 está completamente implementado. Segue o código do contador BCD módulo 10 utilizado como componente:

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity CONTADORBCD10 is
5      port (
6          clock: in std_logic;
7          reset: in std_logic;
8          enable: in std_logic;
9          rci: in std_logic;
10         load: in std_logic;
11         d: in std_logic_vector(3 downto 0);
12         Q: out std_logic_vector(3 downto 0);
13         RCO: out std_logic
14     );
15 end CONTADORBCD10;
16
17 architecture CONTADORBCD10_ARCH of CONTADORBCD10 is
18
19     type estado is (ST0,ST1,ST2,ST3,ST4,ST5,ST6,ST7,ST8,ST9);
20
21     signal currState, nextState : estado;
22     signal nextQ : std_logic_vector(3 downto 0);
23     signal nextRCO : std_logic;
24
25 begin
26
27     sync_proc: process(clock)
28     begin
29         if rising_edge(clock) then
30             currState <= nextState;
31             Q <= nextQ;
32             RCO <= nextRCO;
33         end if;
34     end process sync_proc;
35
36     comb_proc: process(currState, reset, load, d, enable, rci)
37     begin
38         if (reset = '1') then
39             nextState <= ST0;
40             nextQ <= "0000";
41             nextRCO <= '1';
42         elsif (load = '1') then
43             case (d) is
44                 when "0000" =>
45                     nextState <= ST0;
46                     nextQ <= "0000";
47                     nextRCO <= '1';

```

```
48         when "0001" =>
49             nextState <= ST1;
50             nextQ <= "0001";
51             nextRCO <= '1';
52         when "0010" =>
53             nextState <= ST2;
54             nextQ <= "0010";
55             nextRCO <= '1';
56         when "0011" =>
57             nextState <= ST3;
58             nextQ <= "0011";
59             nextRCO <= '1';
60         when "0100" =>
61             nextState <= ST4;
62             nextQ <= "0100";
63             nextRCO <= '1';
64         when "0101" =>
65             nextState <= ST5;
66             nextQ <= "0101";
67             nextRCO <= '1';
68         when "0110" =>
69             nextState <= ST6;
70             nextQ <= "0110";
71             nextRCO <= '1';
72         when "0111" =>
73             nextState <= ST7;
74             nextQ <= "0111";
75             nextRCO <= '1';
76         when "1000" =>
77             nextState <= ST8;
78             nextQ <= "1000";
79             nextRCO <= '1';
80         when "1001" =>
81             nextState <= ST9;
82             nextQ <= "1001";
83             nextRCO <= '0';
84         when others =>
85             nextState <= ST0;
86             nextQ <= "0000";
87             nextRCO <= '1';
88     end case;
```

```

89         elsif (enable = '0') and (rc1 = '0') then
90             case (currState) is
91                 when ST0 =>
92                     nextState <= ST1;
93                     nextQ <= "0001";
94                     nextRCO <= '1';
95                 when ST1 =>
96                     nextState <= ST2;
97                     nextQ <= "0010";
98                     nextRCO <= '1';
99                 when ST2 =>
100                     nextState <= ST3;
101                     nextQ <= "0011";
102                     nextRCO <= '1';
103                 when ST3 =>
104                     nextQ <= "0100";
105                     nextRCO <= '1';
106                     nextState <= ST4;
107                 when ST4 =>
108                     nextState <= ST5;
109                     nextQ <= "0101";
110                     nextRCO <= '1';
111                 when ST5 =>
112                     nextState <= ST6;
113                     nextQ <= "0110";
114                     nextRCO <= '1';
115                 when ST6 =>
116                     nextState <= ST7;
117                     nextQ <= "0111";
118                     nextRCO <= '1';
119                 when ST7 =>
120                     nextState <= ST8;
121                     nextQ <= "1000";
122                     nextRCO <= '1';
123                 when ST8 =>
124                     nextState <= ST9;
125                     nextQ <= "1001";
126                     nextRCO <= '0';
127                 when ST9 =>
128                     nextState <= ST0;
129                     nextQ <= "0000";
130                     nextRCO <= '1';
131                 when others =>
132                     nextState <= ST0;
133                     nextQ <= "0000";
134                     nextRCO <= '1';
135             end case;
136         end if;
137     end process comb_proc;
138 end CONTADORBCD10_ARCH;

```

Listagem 2: Código do contador BCD módulo 10.

SIMULAÇÃO E ANÁLISE

Para o contador módulo 100, serão realizados dois testes. Um que verifica a funcionalidade de contagem (Figuras 3 até 12) e outro que verifica a funcionalidade do load (Figura 13). Inicia-se o teste da contagem com um pequeno pulso em reset

(visualizado na Figura 3), e, em seguida, a permanência de todas as entradas em zero. O resultado da simulação segue nas Figuras 3 até 12. Nota-se, na Figura 12, que a saída RCO corretamente entra em baixa apenas no estado 99.

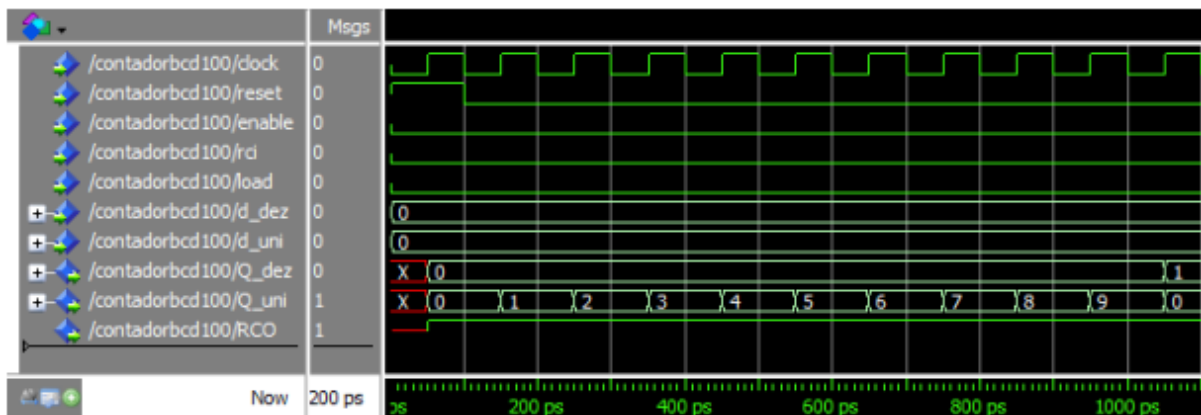


Figura 3: Simulação da contagem do contador 100 quando as dezenas é 0.

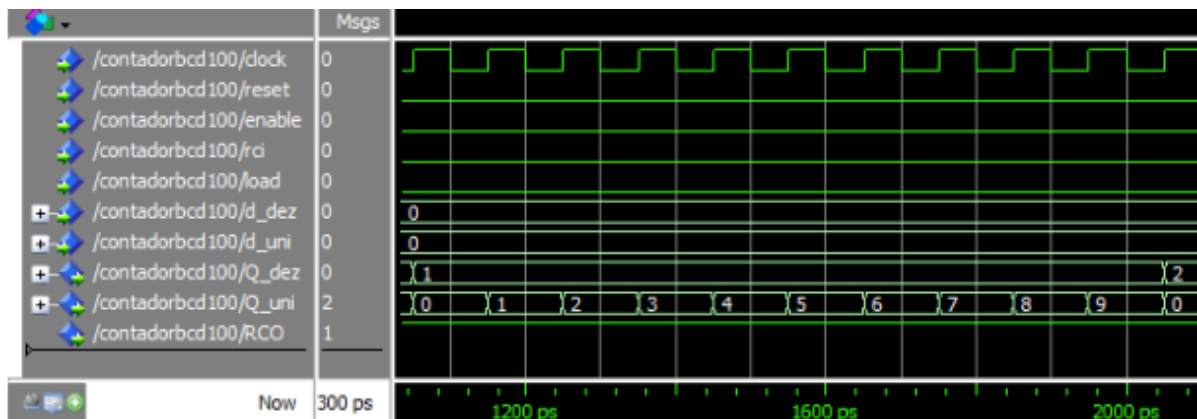


Figura 4: Simulação da contagem do contador 100 quando as dezenas é 1

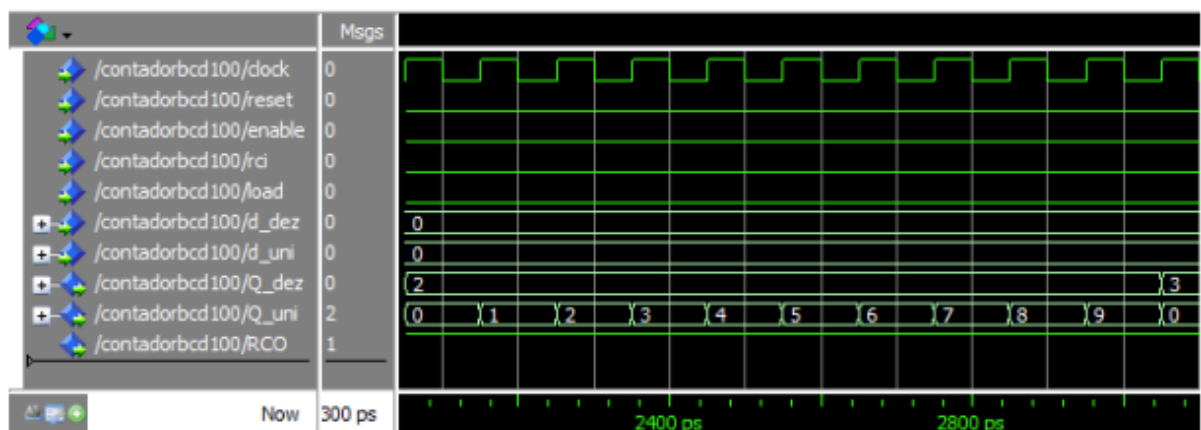


Figura 5: Simulação da contagem do contador 100 quando as dezenas é 2

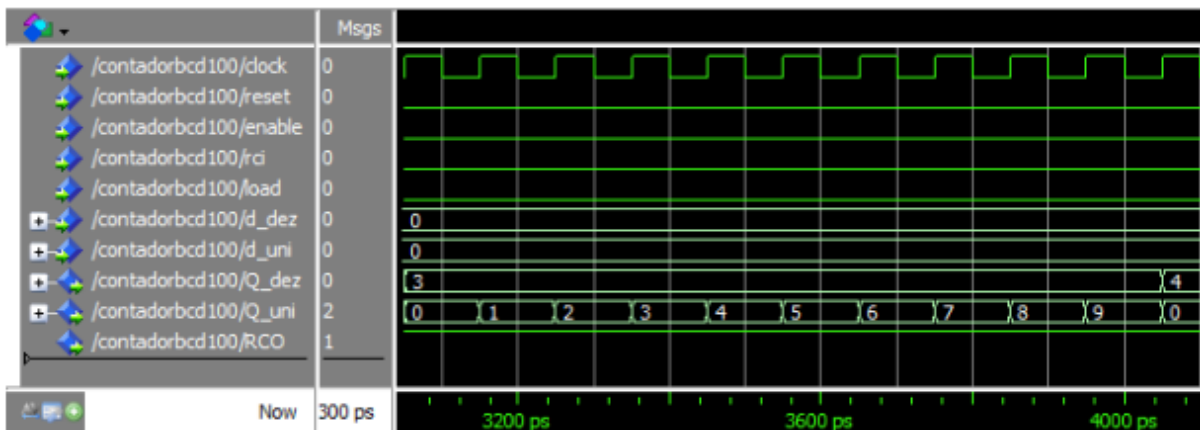


Figura 6: Simulação da contagem do contador 100 quando as dezenas é 3

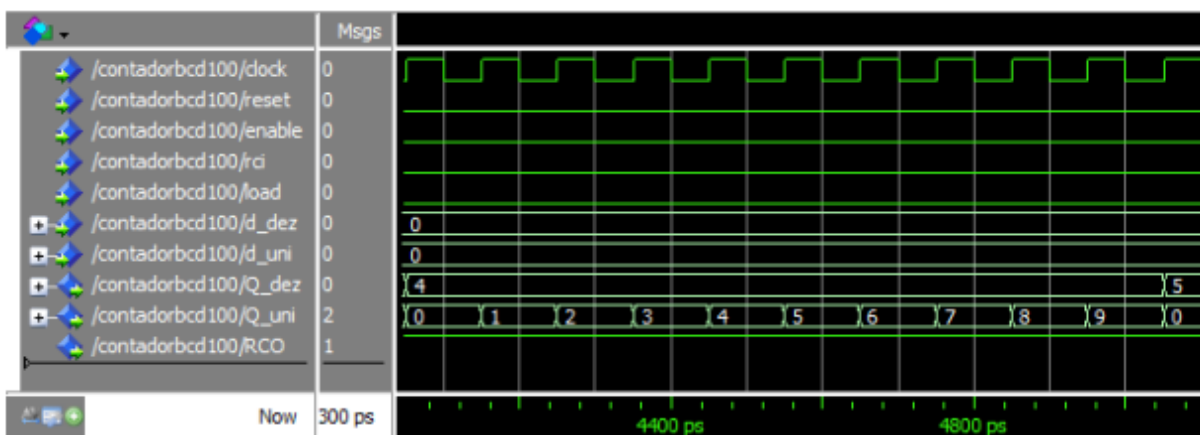


Figura 7: Simulação da contagem do contador 100 quando as dezenas é 4

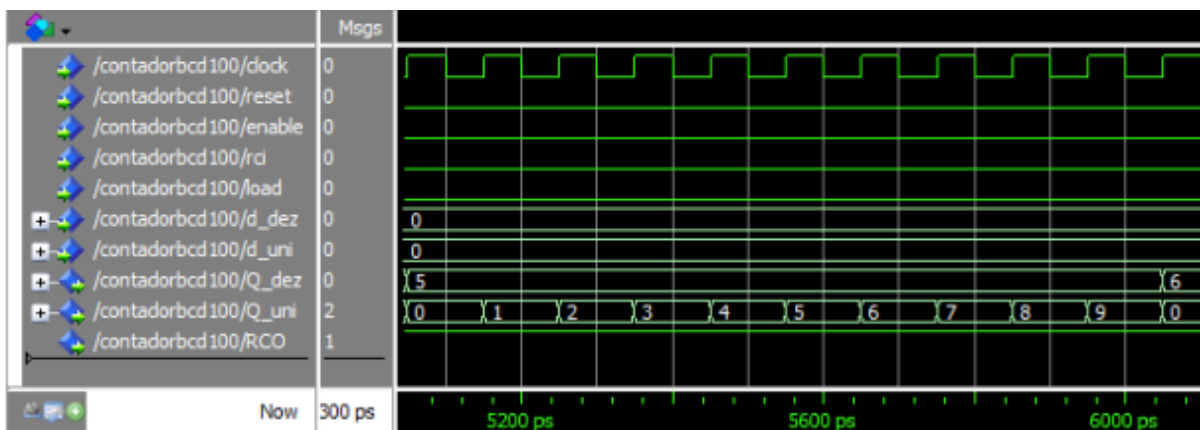


Figura 8: Simulação da contagem do contador 100 quando as dezenas é 5

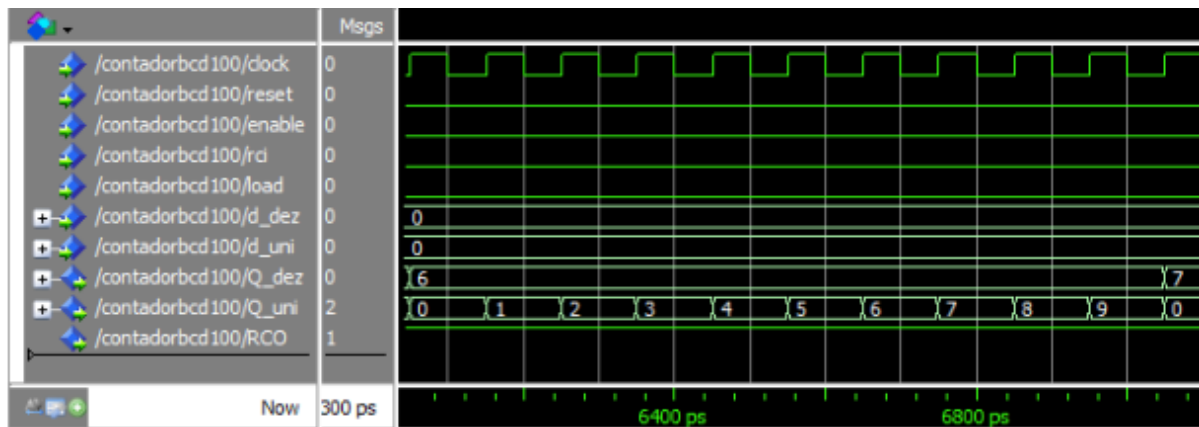


Figura 9: Simulação da contagem do contador 100 quando as dezenas é 6

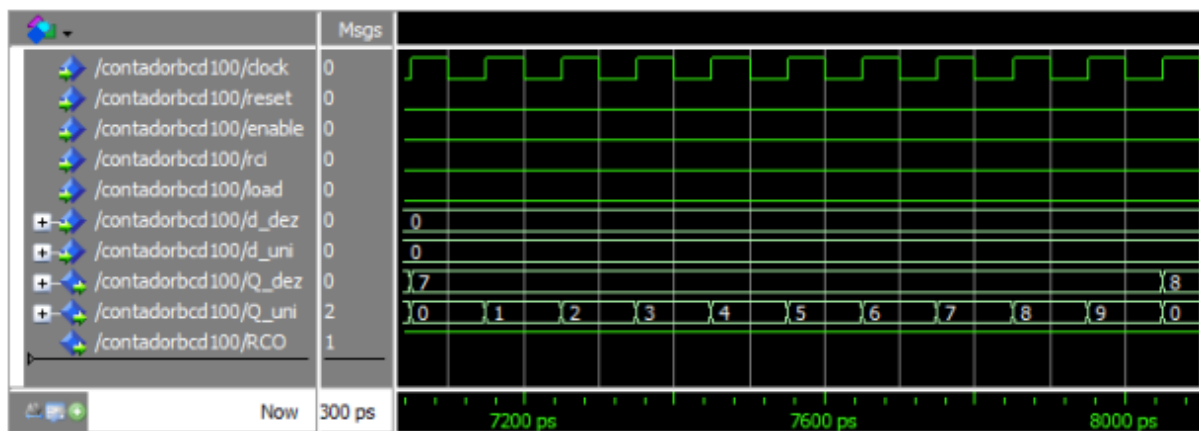


Figura 10: Simulação da contagem do contador 100 quando as dezenas é 7

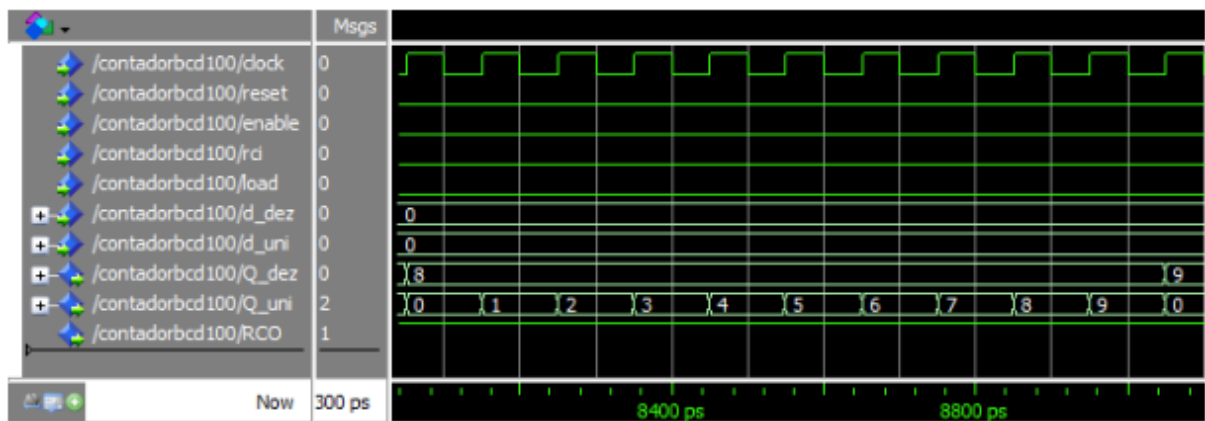


Figura 11: Simulação da contagem do contador 100 quando as dezenas é 8

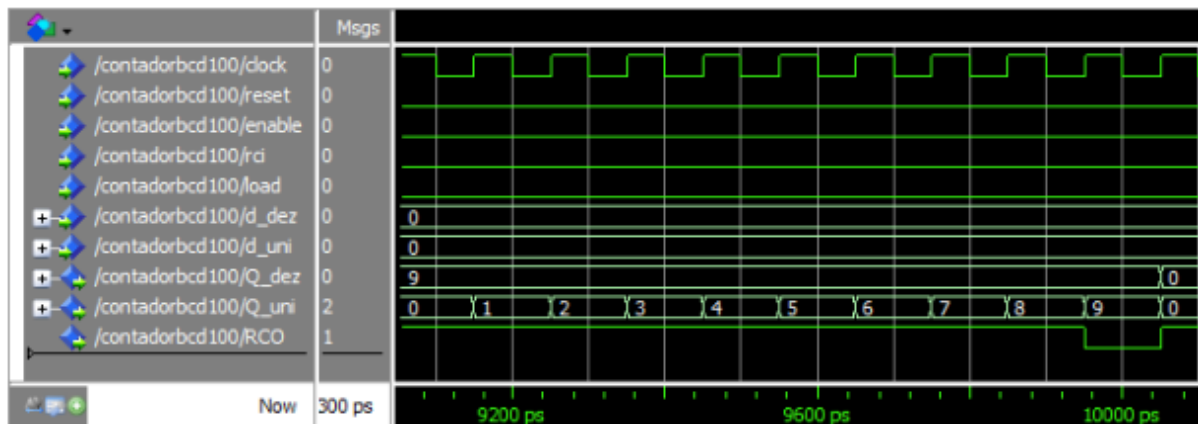


Figura 12: Simulação da contagem do contador 100 quando as dezenas é 9

A simulação da funcionalidade load, na Figura 14 é semelhante ao teste realizado anteriormente, no contador de módulo 100. Inicia-se com um pequeno pulso a reset e, depois, insere-se o estado 25. Verifica-se, com alguns ciclos do clock, que o contador continua em funcionamento normal após o carregamento.

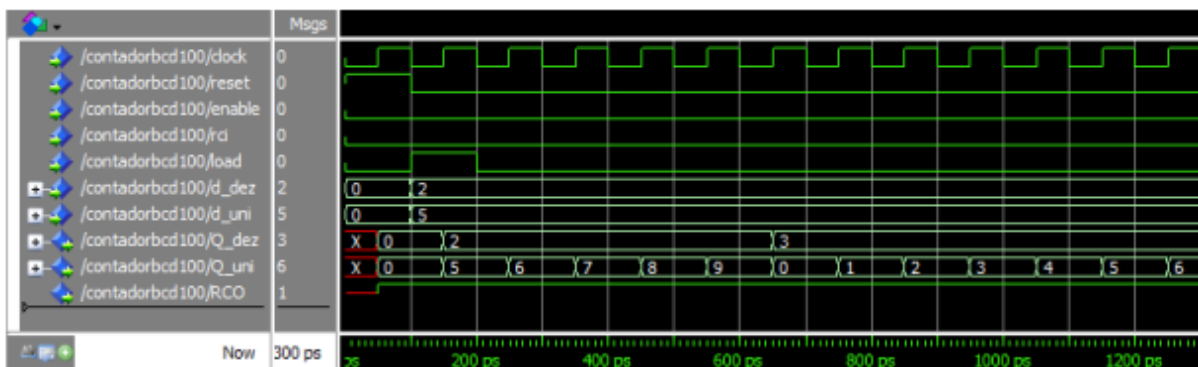


Figura 13: Simulação do load do contador 100.

COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:

```
# Compile of contadorBCD10.vhd was successful.
# Compile of contadorBCD100.vhd was successful.
# 2 compiles, 0 failed with no errors.
```

Figura 15: Compilação dos códigos

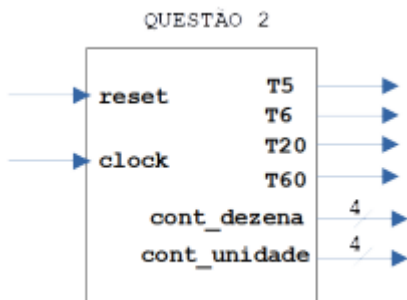
Como visto na figura , nenhum código tem algum erro de sintaxe.

QUESTÃO 2

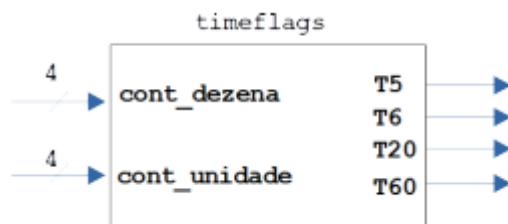
TEORIA

Na segunda questão vamos implementar um sistema de temporização do controle de semáforos, com 'reset' síncrono, clock de 1Hz, quatro saídas binárias, indicando que já se passaram, respectivamente, 5 segundos (T5), 6 segundos (T6), 20 segundos (T20) e 60 segundos (T60). A caixa preta da arquitetura interna desse sistema é apresentada na Figura 1(a). A caixa preta e o diagrama de blocos da arquitetura interna desse sistema são apresentados na Figura 1.

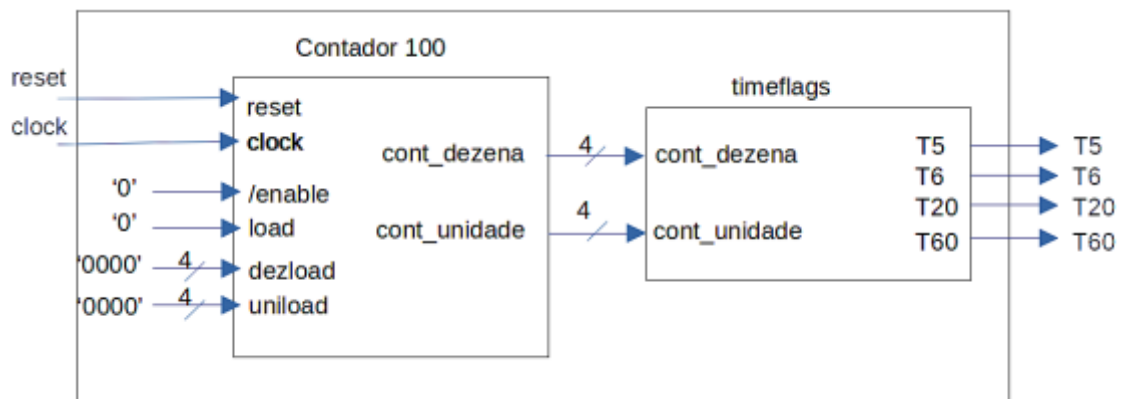
a)



b)



c)



- **Figura 1:** Caixa preta (a) e diagrama de blocos da arquitetura interna (b) e sistema a ser implementado na questão 2 (c).

Na Figura 1(c) é apresentado o sistema completo a ser implementado. A entidade timeflags (Figura 1b) é responsável por verificar se já se passaram, respectivamente, 5 segundos (T5), 6 segundos (T6), 20 segundos (T20) e 60 segundos (T60). As entradas são dois números em representação BCD, dezena e unidade, e as saídas são os quatro flags indicadores: T5, T6, T20 e T60.

CÓDIGO

Esses flags indicadores podem ser implementados usando atribuições condicionais (estrutura “when-else”) e os operadores de comparação da linguagem VHDL. Trata-se de um circuito puramente combinacional, não sendo necessário o uso de uma estrutura “process”. Portanto segue a implementação do timeflags:

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity timeflags is
5      port (
6          dezena : in  std_logic_vector(3 downto 0);
7          unidade : in  std_logic_vector(3 downto 0);
8          T5      : out std_logic;
9          T6      : out std_logic;
10         T20     : out std_logic;
11         T60     : out std_logic
12     );
13 end timeflags;
14
15 architecture behavior of timeflags is
16     -- Sinal interno de 8 bits para juntar a dezena e a unidade
17     signal tempo_bcd : std_logic_vector(7 downto 0);
18 begin
19
20     -- Concatena os 4 bits da dezena com os 4 bits da unidade
21     tempo_bcd <= dezena & unidade;
22
23     T5 <= '1' when tempo_bcd = x"05" else '0';
24     T6 <= '1' when tempo_bcd = x"06" else '0';
25     T20 <= '1' when tempo_bcd = x"20" else '0';
26     T60 <= '1' when tempo_bcd = x"60" else '0';
27
28 end behavior;
```

Listagem 1 : Código do timeflags

O código da Listagem 1 implementa as bibliotecas necessárias (linha 1 e 2), e as portas anteriormente citadas (linhas 4 a 13).

Já para a arquitetura do timeflags. Basta, primeiramente, definir um sinal de 8 bits, tempo_bcd, na linha 17, para concatenar a dezena e a unidade, onde isso é

feito na linha 21 deixando assim a dezena e a unidade em uma única variável. Por fim, realiza a lógica da saída de cada T utilizando when e else para quando o tempo for o certo sair 1 na saída caso não seja manter em 0. Dessa forma, o contador timeflashes está implementado. Assim podemos implementar o sistema completo:

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity sistema_semaforo is
5      port (
6          clock : in  std_logic;
7          reset : in  std_logic;
8          T5    : out std_logic;
9          T6    : out std_logic;
10         T20   : out std_logic;
11         T60   : out std_logic
12     );
13 end sistema_semaforo;
14
15 architecture arch of sistema_semaforo is
16     component CONTADORBCD100 is
17         port (
18             clock: in std_logic;
19             reset: in std_logic;
20             enable: in std_logic;
21             rci: in std_logic;
22             load: in std_logic;
23             d_uni: in std_logic_vector(3 downto 0);
24             d_dez: in std_logic_vector(3 downto 0);
25             Q_uni: out std_logic_vector(3 downto 0);
26             Q_dez: out std_logic_vector(3 downto 0);
27             RCO: out std_logic
28         );
29     end component;
30
31     component timeflashes is
32         port (
33             dezena : in  std_logic_vector(3 downto 0);
34             unidade : in  std_logic_vector(3 downto 0);
35             T5     : out std_logic;
36             T6     : out std_logic;
37             T20    : out std_logic;
38             T60    : out std_logic
39         );
40     end component;
41
```

```

42      -- Sinais internos para interconectar o Contador ao Timeflags
43      signal sig_uni : std_logic_vector(3 downto 0);
44      signal sig_dez : std_logic_vector(3 downto 0);
45      signal sig_rco : std_logic;
46
47  begin
48
49
50      inst_contador: CONTADORBCD100
51      port map (
52          clock => clock,
53          reset => reset,
54          enable => '0',          -- Sempre habilitado para contar
55          rci   => '0',          -- Ripple Carry Input ativo
56          load  => '0',          -- Desabilita a carga paralela
57          d_uni => "0000",       -- Entradas de dados zeradas
58          d_dez => "0000",
59          Q_uni => sig_uni,       -- Conecta à unidade do timeflags
60          Q_dez => sig_dez,       -- Conecta à dezena do timeflags
61          RCO  => sig_rco
62      );
63
64      -- Instanciação do bloco de Flags de Tempo
65      inst_flags: timeflags
66      port map (
67          dezena => sig_dez,
68          unidade => sig_uni,
69          T5     => T5,
70          T6     => T6,
71          T20    => T20,
72          T60    => T60
73      );
74
75  end arch;

```

- **Listagem 2** : Código do sistema do semáforo

O código da Listagem 2 implementa as bibliotecas necessárias (linha 1 e 2), e as portas anteriormente citadas (linhas 4 a 13).

Já para a arquitetura do timeflags. Basta, primeiramente, definir os componentes timeflags e o contador 100 implementados anteriormente (linhas 16 a 40). Em seguida, definir 2 sinais de 4 bits um para a unidade e outro para a dezena e um sinal para o rco (linhas 43 a 45). Por fim, realiza as conexões e ligações da entre o contador e o timeflags. Dessa forma, o sistema está implementado. Assim podemos simular o sistema completo, para isso foi criado o seguinte código para simular:

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity sistema_tempo_tb is
5  -- Testbench não possui portas externas
6  end sistema_tempo_tb;
7
8  architecture sim of sistema_tempo_tb is
9
10     -- Componente 1: Contador BCD 100
11     component CONTADORBCD100 is
12         port (
13             clock : in  std_logic;
14             reset  : in  std_logic;
15             enable : in  std_logic;
16             rci    : in  std_logic;
17             load   : in  std_logic;
18             d_uni  : in  std_logic_vector(3 downto 0);
19             d_dez  : in  std_logic_vector(3 downto 0);
20             Q_uni  : out std_logic_vector(3 downto 0);
21             Q_dez  : out std_logic_vector(3 downto 0);
22             RCO    : out std_logic
23         );
24     end component;
25
26     -- Componente 2: Timeflags
27     component timeflags is
28         port (
29             dezena : in  std_logic_vector(3 downto 0);
30             unidade : in  std_logic_vector(3 downto 0);
31             T5     : out std_logic;
32             T6     : out std_logic;
33             T20    : out std_logic;
34             T60    : out std_logic
35         );
36     end component;
37
38     -- Sinais de teste (Testbench)
39     signal clk_tb    : std_logic := '0';
40     signal rst_tb    : std_logic := '0';
41     signal q_uni_tb  : std_logic_vector(3 downto 0);
42     signal q_dez_tb  : std_logic_vector(3 downto 0);
43     signal dummy_rco : std_logic;

```



```

44
45     -- Saídas dos sinalizadores de tempo que queremos observar
46     signal T5_tb      : std_logic;
47     signal T6_tb      : std_logic;
48     signal T20_tb     : std_logic;
49     signal T60_tb     : std_logic;
50
51     -- 1 ciclo de clock = 10 ns (representando 1 segundo na simulação)
52     constant CLK_PERIOD : time := 10 ns;
53
54     begin
55
56         -- Instanciação do Contador Módulo 100
57         U1_Contador: CONTADORBCD100
58             port map (
59                 clock => clk_tb,
60                 reset => rst_tb,
61                 enable => '0',          -- Ativo em baixo: Garante contagem contínua
62                 rci    => '0',          -- Ativo em baixo: Garante contagem contínua
63                 load   => '0',          -- Desativado
64                 d_uni  => "0000",
65                 d_dez  => "0000",
66                 Q_uni  => q_uni_tb,
67                 Q_dez  => q_dez_tb,
68                 RCO    => dummy_rco
69             );
70
71         -- Instanciação do Bloco de Flags conectado nas saídas do contador
72         U2_Timeflags: timeflags
73             port map (
74                 dezena => q_dez_tb,
75                 unidade => q_uni_tb,
76                 T5     => T5_tb,
77                 T6     => T6_tb,
78                 T20    => T20_tb,
79                 T60    => T60_tb
80             );
81
82         -- Processo Gerador de Clock
83         clk_process : process
84         begin
85             while true loop

```

```

85         while true loop
86             clk_tb <= '0';
87             wait for CLK_PERIOD / 2;
88             clk_tb <= '1';
89             wait for CLK_PERIOD / 2;
90         end loop;
91     end process;
92
93     -- Processo de Estímulo
94     stim_process : process
95     begin
96         -- Dá um pulso inicial de Reset para zerar o contador
97         rst_tb <= '1';
98         wait for CLK_PERIOD * 1.5;
99         rst_tb <= '0';
100
101         -- Agora deixamos o contador rodar livremente por 65 ciclos de clock
102         -- para dar tempo de passar por todas as metas (5s, 6s, 20s e 60s).
103         wait for CLK_PERIOD * 65;
104
105         -- Finaliza a simulação
106         wait;
107     end process;
108
109 end sim;

```

- **Listagem 3:** Código para simular o sistema do semáforo.

SIMULAÇÃO E ANÁLISE

Com o testbench obtemos a seguinte onda:

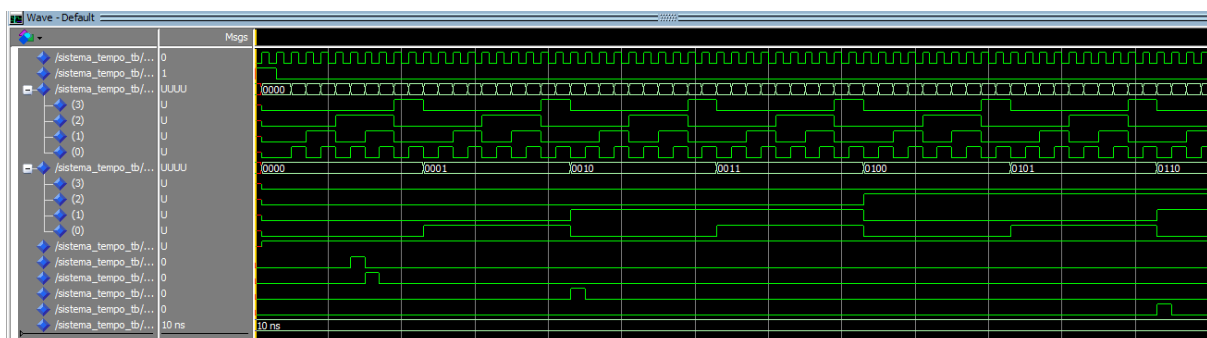
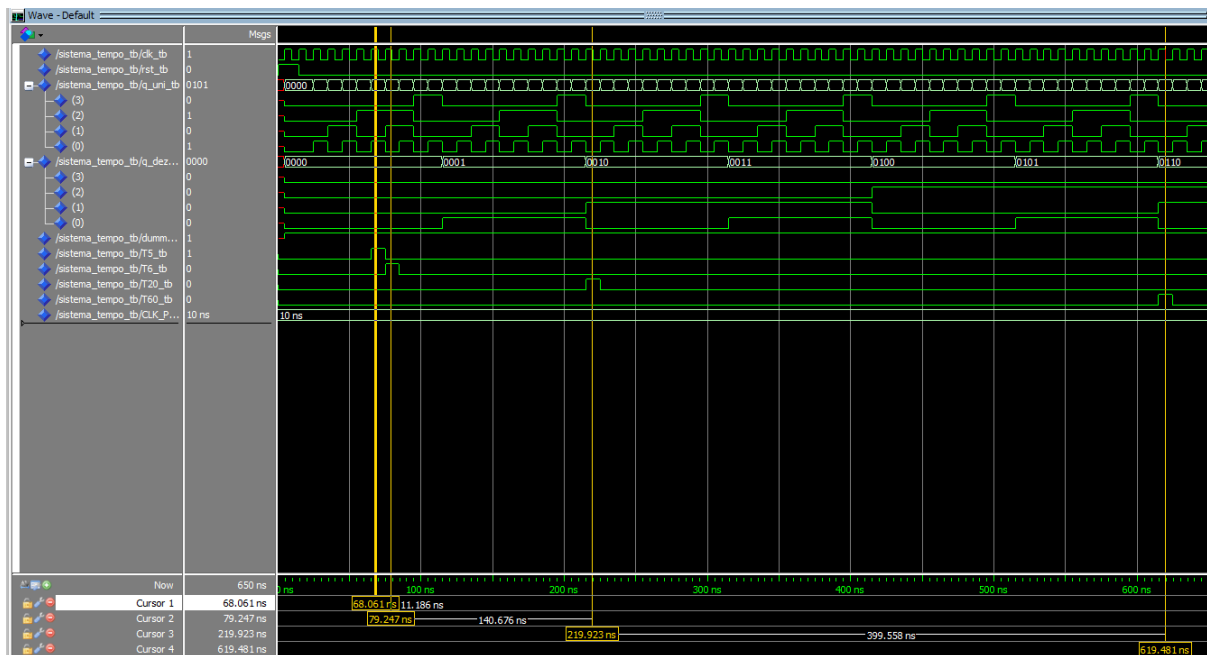


Figura 2: simulação questão 2



- O sinal **reset** cai para '0'. O contador é liberado e começa a incrementar a cada borda de subida do clock.

Em 65 ns — Detecção de 5 Segundos (**T5**)

- O contador atinge o valor decimal **05** (**Q_dez** = "0000" e **Q_uni** = "0101").
- O flag **T5** sobe para '1' e permanece ativo até 75 ns

Em 75 ns — Detecção de 6 Segundos (**T6**)

- O contador avança para **06** (**Q_dez** = "0000" e **Q_uni** = "0110").
- O flag **T5** volta para '0' e o flag **T6** sobe para '1', permanecendo ativo até 85 ns.

Em 215 ns — Detecção de 20 Segundos (**T20**)

- O contador atinge o valor **20** (**Q_dez** = "0010" e **Q_uni** = "0000").
- O flag **T20** sobe para '1' e permanece ativo até 225 ns.

Em 615 ns — Detecção de 60 Segundos (**T60**)

- O contador atinge o valor máximo do teste, **60** (**Q_dez** = "0110" e **Q_uni** = "0000").
- O flag **T60** sobe para '1' e permanece ativo até 625 ns.

COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:

```
# Compile of CONTADORBCD10.vhd was successful.  
# Compile of CONTADORBCD100.vhd was successful.  
# Compile of sistema_semaforo.vhd was successful.  
# Compile of tb_sistema.vhd was successful.  
# Compile of timeflags.vhd was successful.  
# 5 compiles, 0 failed with no errors.  
  
ModelSim>
```

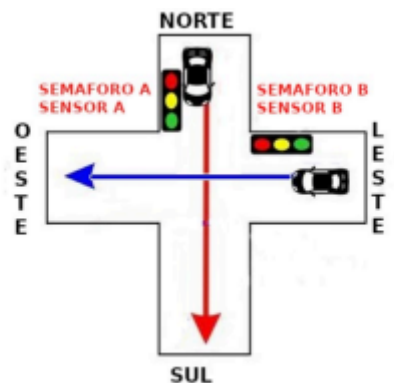
Figura 15: Compilação dos códigos

Como visto na figura , nenhum código tem algum erro de sintaxe.

QUESTÃO 3

TEORIA

Na terceira questão vamos implementar o sistema de controle de semáforos com três bits de entrada (sensor de carros na direção norte/sul, sensor de carros na direção leste/oeste e chave de liga/desliga do sistema) e saídas representando os estados dos semáforos e o temporizador/contador (luzes verde, amarela e vermelha de cada direção e estado do contador). O cruzamento é ilustrado na Figura 1:



- **Figura 1:** Ilustração do cruzamento, mostrando a posição dos semáforos e dos sensores.

O cruzamento deverá ficar liberado para determinada direção (luz verde) e bloqueado na outra direção (luz vermelha) por 60 segundos, exceto caso haja carro esperando na direção oposta e não haja carros atravessando na direção em questão e já tenha se passado pelo menos 20 segundos. Antes de trocar a pista liberada, o semáforo deverá mostrar a luz amarela (com luz vermelha no outro

semáforo) por 6 segundos e, em seguida, a luz vermelha (em ambos os semáforos) por 5 segundos. A qualquer momento, se a chave de liga/desliga for desativada, os semáforos deverão entrar em estado intermitente, no qual alternarão entre a luz amarela e nenhuma luz acesa, com intervalo de 1 segundo entre cada, devendo voltar a qualquer momento ao funcionamento normal caso a chave seja reativada.

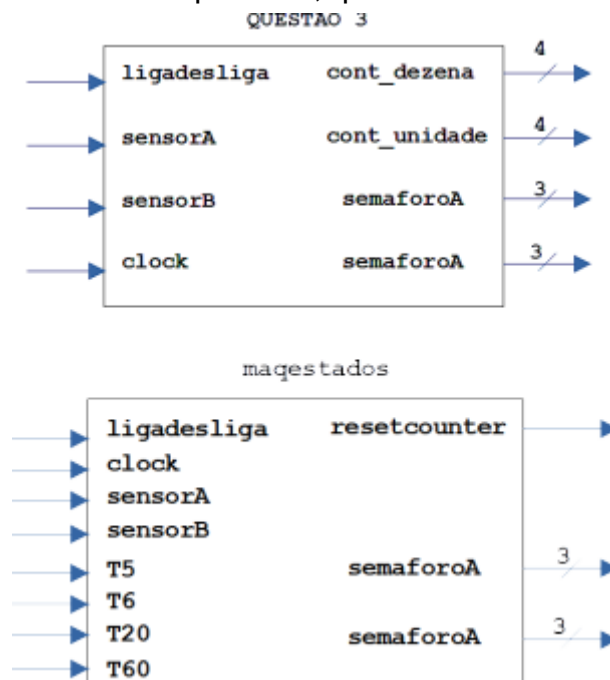
Mais especificamente, o sistema deve ser implementado com as seguintes entradas:

- sensorA: 1 bit que indica se há carros na direção norte/sul),
- sensorB: 1 bit que indica se há carros na direção leste/oeste),
- Chave ligadesliga: 1 bit que indica se chave de liga/desliga do sistema foi acionada

e as seguintes saídas:

- semaforoA: 3 bits indicando quais das luzes (vermelha, verde e amarela) do semáforo da direção norte/sul estão acesas;
- semaforoB: 3 bits indicando quais das luzes (vermelha, verde e amarela) do semáforo da direção leste/oeste estão acesas;
- contador de tempo: 4 bits para representar as unidades e 4 bits para representar as dezenas do contador de tempo implementado nas questões anteriores.

A caixa preta desse sistema é apresentada na Figura 2. Serão usadas (como componentes) todas as entidades utilizadas na arquitetura interna da entidade da questão 2 e uma nova entidade: maqestados, que será detalhada a seguir.



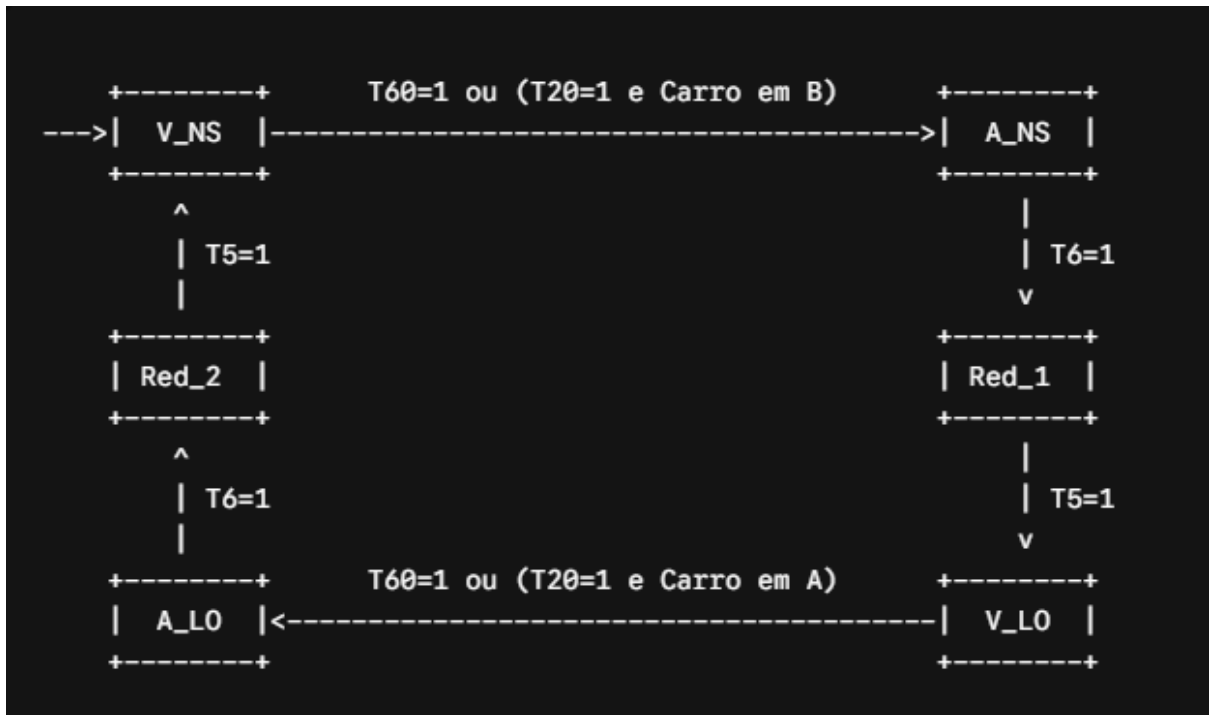
- **Figura 2:** Caixas pretas das entidades exp8visto3 (a) e entidade maqestados (b).

A entidade maqestados é a responsável por implementar o controle dos semáforos. A caixa preta dessa entidade é apresentada na Figura 2b. Deve ser implementada como uma máquina de estados, com duas saídas do tipo Moore — dois vetores de 3 bits semaforoA e semaforoB, que indicarão se as luzes vermelho, amarelo e verde dos semáforos das pistas norte/sul e leste/oeste, respectivamente, estarão acesas ou não (1 bit para cada cor, nessa ordem) — e uma saída do tipo Mealy — um sinal de 1 bit resetcounter, que indicará quando o contador de tempo deverá ser reinicializado. As transições de estado serão controladas pelas variáveis de entrada ligadesliga, sensorA, sensorB, T5, T6, T20 e T60. As três primeiras serão associadas às entradas de mesmo nome da entidade “top level” (Figura 2(a)) e as quatro últimas às saídas da entidade timeflags.

O cruzamento deverá ficar liberado para determinada direção (luz verde) e bloqueado na outra direção (luz vermelha) por 60 segundos, exceto caso haja carro esperando na direção oposta e não haja carros atravessando na direção em questão e já tenha se passado pelo menos 20 segundos. Antes de trocar a pista liberada, o semáforo deverá mostrar a luz amarela (com luz vermelha no outro semáforo) por 6 segundos e, em seguida, a luz vermelha (em ambos os semáforos) por 5 segundos.

A qualquer momento, se a chave de liga/desliga for desativada, os semáforos deverão entrar em estado intermitente, no qual alternarão entre a luz amarela e nenhuma luz acesa, com intervalo de 1 segundo entre cada, devendo voltar a qualquer momento ao funcionamento normal caso a chave seja reativada. Note que a entidade maqestados deve, por meio da saída resetcounter, reinicializar o contador de tempo (implementado pela entidade contador100), isto é, fazer a saída resetcounter = ‘1’, sempre que a máquina estiver se preparando para mudar de estado, isto é, se o estado seguinte (nextState) for diferente do estado atual (currentState). Note que esta é uma saída do tipo Mealy, pois depende não só do estado atual, mas também das variáveis de entrada (que efetivamente determinam o estado seguinte). Isto pode ser implementado com uma lógica combinacional de estado seguinte, atribuindo um valor (‘0’ ou ‘1’) a resetcounter sempre que um valor for atribuído à variável de estado seguinte (nextState).

Para isso foi criado o seguinte fluxo de estados:



CÓDIGO

A entidade **maestados** é o "cérebro" do semáforo. Ela foi projetada como uma **Máquina de Estados Finitos (FSM)** que dita qual cor deve acender em cada cruzamento e quando o tempo deve ser zerado.. Portanto segue a implementação da mesma:

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity maquestados is
5      port (
6          clock      : in  std_logic;
7          reset      : in  std_logic;
8          ligadesliga : in  std_logic;
9          sensorA     : in  std_logic; -- Norte/Sul
10         sensorB     : in  std_logic; -- Leste/Oeste
11         T5          : in  std_logic;
12         T6          : in  std_logic;
13         T20         : in  std_logic;
14         T60         : in  std_logic;
15         semaforoA    : out std_logic_vector(2 downto 0); -- V, A, G
16         semaforoB    : out std_logic_vector(2 downto 0); -- V, A, G
17         resetcounter : out std_logic
18     );
19 end maquestados;
20
21 architecture behavior of maquestados is
22     type state_type is (V_NS, A_NS, Red_1, V_LO, A_LO, Red_2, PISCA_ON, PISCA_OFF);
23     signal current_state, next_state : state_type;
24 begin
25
26     -- Processo Síncrono: Transição de Estado
27     process(clock, reset)
28     begin
29         if reset = '1' then
30             current_state <= V_NS;
31         elsif rising_edge(clock) then
32             current_state <= next_state;
33         end if;
34     end process;
35
36     -- Processo Combinacional: Lógica do Próximo Estado e Saída Mealy (resetcounter)
37     process(current_state, ligadesliga, sensorA, sensorB, T5, T6, T20, T60)
38     begin
39         -- Valores padrão por omissão
40         next_state <= current_state;
41         resetcounter <= '0';
42

```



```

43      -- Verificação global da chave liga/desliga
44      if ligadesliga = '0' then
45          if current_state = PISCA_ON then
46              next_state <= PISCA_OFF;
47              resetcounter <= '1'; -- Força reset síncrono a cada 1s para alternar
48          elsif current_state = PISCA_OFF then
49              next_state <= PISCA_ON;
50              resetcounter <= '1';
51          else
52              next_state <= PISCA_ON;
53              resetcounter <= '1';
54          end if;
55      else
56          -- Funcionamento Normal do Semáforo
57          case current_state is
58              when PISCA_ON | PISCA_OFF =>
59                  next_state <= V_NS;
60                  resetcounter <= '1';
61
62              when V_NS =>
63                  -- Muda após 60s OU (após 20s se houver carro em B e nenhum em A)
64                  if T60 = '1' or (T20 = '1' and sensorB = '1' and sensorA = '0') then
65                      next_state <= A_NS;
66                      resetcounter <= '1';
67                  end if;
68
69              when A_NS =>
70                  if T6 = '1' then
71                      next_state <= Red_1;
72                      resetcounter <= '1';
73                  end if;
74
75              when Red_1 =>
76                  if T5 = '1' then
77                      next_state <= V_LO;
78                      resetcounter <= '1';
79                  end if;
80
81              when V_LO =>
82                  -- Muda após 60s OU (após 20s se houver carro em A e nenhum em B)
83                  if T60 = '1' or (T20 = '1' and sensorA = '1' and sensorB = '0') then
84                      next_state <= A_LO;

```

```

85         resetcounter <= '1';
86     end if;
87
88     when A_LO =>
89         if T6 = '1' then
90             next_state <= Red_2;
91             resetcounter <= '1';
92         end if;
93
94     when Red_2 =>
95         if T5 = '1' then
96             next_state <= V_NS;
97             resetcounter <= '1';
98         end if;
99
100    when others =>
101        next_state <= V_NS;
102    end case;
103 end if;
104 end process;
105
106 -- Saídas Moore: Dependentes apenas do Estado Atual (Luzes Vermelho, Amarelo, Verde)
107 process(current_state)
108 begin
109     case current_state is
110         when V_NS =>
111             semaforoA <= "001"; -- Verde
112             semaforoB <= "100"; -- Vermelho
113         when A_NS =>
114             semaforoA <= "010"; -- Amarelo
115             semaforoB <= "100"; -- Vermelho
116         when Red_1 =>
117             semaforoA <= "100"; -- Vermelho
118             semaforoB <= "100"; -- Vermelho
119         when V_LO =>
120             semaforoA <= "100"; -- Vermelho
121             semaforoB <= "001"; -- Verde
122         when A_LO =>
123             semaforoA <= "100"; -- Vermelho
124             semaforoB <= "010"; -- Amarelo
125         when Red_2 =>

```

```

126         semaforoA <= "100"; -- Vermelho
127         semaforoB <= "100"; -- Vermelho
128         when PISCA_ON =>
129             semaforoA <= "010"; -- Amarelo piscando
130             semaforoB <= "010"; -- Amarelo piscando
131         when PISCA_OFF =>
132             semaforoA <= "000"; -- Apagado
133             semaforoB <= "000"; -- Apagado
134         when others =>
135             semaforoA <= "100";
136             semaforoB <= "100";
137         end case;
138     end process;
139
140 end behavior;

```

- **Listagem 1** : Código da máquina de estados

O código da Listagem 1 implementa as bibliotecas necessárias (linhas 1 e 2) e as portas anteriormente citadas para o controle do semáforo (linhas 4 a 19).

Já para a arquitetura da **maqestados**, basta primeiramente definir um tipo enumerado contendo todos os estados do semáforo na linha 22 (**V_NS**, **A_NS**, **Red_1**, **V_LO**, **A_LO**, **Red_2**, **PISCA_ON**, **PISCA_OFF**), além de criar os sinais internos **current_state** e **next_state** na linha 23 para gerenciar a transição da máquina.

O funcionamento é dividido em três processos principais:

- **Processo Síncrono (**sync_proc**, linhas 27 a 34)**: É responsável por atualizar o estado atual na borda de subida do clock ou forçar o sistema para o estado inicial **V_NS** caso o **reset** geral seja acionado.
- **Lógica de Próximo Estado (**process** combinacional, linhas 36 a 104)**: Avalia as condições de trânsito. Primeiramente, verifica a chave **ligadesliga**; se estiver em '0', a máquina é forçada a alternar entre os estados de emergência **PISCA_ON** e **PISCA_OFF** a cada segundo. Se estiver em '1', o fluxo segue o ciclo normal utilizando uma estrutura **case**. Para os estados verdes (**V_NS** e **V_LO**), o sistema testa o tempo regulamentar de 60 segundos (**T60**) ou a exceção de corte de pista após 20 segundos (**T20**) caso haja carros esperando no sensor oposto e a pista atual esteja vazia. Para as transições de amarelo e vermelho duplo, o sistema aguarda os flags **T6** e **T5**, respectivamente. É neste bloco que a saída do tipo Mealy **resetcounter** é

acionada com '1' sempre que uma mudança de estado é detectada, garantindo que o contador BCD retorne para zero.

- **Lógica de Saída Moore (linhas 107 a 138):** Avalia puramente o estado atual do semáforo para acender as lâmpadas correspondentes. As saídas `semaforoA` e `semaforoB` recebem vetores de 3 bits que representam, nesta ordem, as luzes Vermelha, Amarela e Verde. Por exemplo, no estado `V_NS`, a pista Norte/Sul recebe "`001`" (Verde) e a Leste/Oeste recebe "`100`" (Vermelho). Nos estados de pisca, as luzes alternam entre amarelo "`010`" e totalmente apagadas "`000`".

Dessa forma, a máquina de estados está implementada. Com isso podemos implementar a questão 3 completamente:

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity exp8visto3 is
5      port (
6          clock      : in  std_logic;
7          reset       : in  std_logic;
8          ligadesliga : in  std_logic;
9          sensorA     : in  std_logic;
10         sensorB     : in  std_logic;
11         semaforoA    : out std_logic_vector(2 downto 0);
12         semaforoB    : out std_logic_vector(2 downto 0);
13         contador_uni : out std_logic_vector(3 downto 0); -- Unidades do display
14         contador_dez : out std_logic_vector(3 downto 0)  -- Dezenas do display
15     );
16 end exp8visto3;
17
18 architecture structural of exp8visto3 is
19
20     -- Componente Contador 100 fornecido
21     component CONTADORBCD100 is
22         port (
23             clock: in std_logic;
24             reset: in std_logic;
25             enable: in std_logic;
26             rci: in std_logic;
27             load: in std_logic;
28             d_uni: in std_logic_vector(3 downto 0);
29             d_dez: in std_logic_vector(3 downto 0);
30             Q_uni: out std_logic_vector(3 downto 0);
31             Q_dez: out std_logic_vector(3 downto 0);
32             RCO: out std_logic
33         );
34     end component;
35
36     -- Componente Timeflags criado na questão 2
37     component timeflags is
38         port (
39             dezena : in  std_logic_vector(3 downto 0);
40             unidade : in  std_logic_vector(3 downto 0);
41             T5      : out std_logic;
42             T6      : out std_logic;
43             T20     : out std_logic;

```

```

43         T20      : out std_logic;
44         T60      : out std_logic
45     );
46 end component;
47
48 -- Componente Máquina de Estados criado nesta questão
49 component maestados is
50     port (
51         clock      : in  std_logic;
52         reset      : in  std_logic;
53         ligadesliga : in  std_logic;
54         sensorA    : in  std_logic;
55         sensorB    : in  std_logic;
56         T5         : in  std_logic;
57         T6         : in  std_logic;
58         T20        : in  std_logic;
59         T60        : in  std_logic;
60         semaforoA  : out std_logic_vector(2 downto 0);
61         semaforoB  : out std_logic_vector(2 downto 0);
62         resetcounter : out std_logic
63     );
64 end component;
65
66 -- Sinais internos de interconexão
67 signal q_unidades, q_dezenas : std_logic_vector(3 downto 0);
68 signal w_T5, w_T6, w_T20, w_T60 : std_logic;
69 signal w_resetcounter           : std_logic;
70 signal combo_reset_contador     : std_logic;
71 signal dummy_rco                : std_logic;
72
73 begin
74
75     -- O contador deve resetar caso ocorra o Reset Geral OU se a FSM pedir para resetar
76     combo_reset_contador <= reset or w_resetcounter;
77
78     -- Envia a contagem atual diretamente para as saídas do sistema do Display
79     contador_uni <= q_unidades;
80     contador_dez <= q_dezenas;
81

```

```

83     U1_Contador: CONTADORBCD100
84         port map (
85             clock => clock,
86             reset => combo_reset_contador, -- Reset controlado pelo sistema e FSM
87             enable => '0',
88             rci    => '0',
89             load   => '0',
90             d_uni  => "0000",
91             d_dez  => "0000",
92             Q_uni  => q_unidades,
93             Q_dez  => q_dezenas,
94             RCO    => dummy_rco
95         );
96
97     -- Instanciação dos Timeflags (Identificadores de tempo)
98     U2_Timeflags: timeflags
99         port map (
100             dezena => q_dezenas,
101             unidade => q_unidades,
102             T5     => w_T5,
103             T6     => w_T6,
104             T20    => w_T20,
105             T60    => w_T60
106         );
107
108     -- Instanciação do Bloco de Controle (Máquina de Estados)
109     U3_MaqEstados: maqestados
110         port map (
111             clock      => clock,
112             reset      => reset,
113             ligadesliga => ligadesliga,
114             sensorA    => sensorA,
115             sensorB    => sensorB,
116             T5         => w_T5,
117             T6         => w_T6,
118             T20        => w_T20,
119             T60        => w_T60,
120             semaforoA  => semaforoA,
121             semaforoB  => semaforoB,
122             resetcounter => w_resetcounter
123         );

```

Listagem 2: Código final do problema dos semáforos

O código da Listagem 2 implementa as bibliotecas necessárias (linhas 1 e 2) e as portas da caixa preta do sistema completo (linhas 4 a 16), que incluem as entradas dos sensores, chave e clock, e as saídas para os semáforos e os displays do contador.

Já para a arquitetura estrutural, primeiramente declaram-se os três blocos internos como componentes (linhas 19 a 55): o `CONTADORBCD100`, o bloco combinacional `timeflags` e o controlador de tráfego `majestados`. Na sequência, são definidos os sinais internos de interconexão (linhas 58 a 62) para transportar as contagens de dezenas e unidades, os quatro flags de tempo e o sinal de reset dinâmico.

A lógica de interconexão e funcionamento do sistema ocorre da seguinte forma nas linhas de execução:

- Na linha 66, é feita uma lógica `OR` combinacional para gerar o sinal `combo_reset_contador`. Isso garante que o contador módulo 100 seja reinicializado tanto por um `reset` manual do usuário quanto pelo sinal automático `resetcounter` enviado pela máquina de estados.
- Nas linhas 69 e 70, as saídas de contagem das unidades e dezenas do contador são jogadas diretamente para as portas de saída do sistema para atualizar os displays.
- Na linha 73, o componente `CONTADORBCD100` é instanciado. Para adequar o sistema à nova lógica ativa em baixo do contador, as entradas de controle `enable` e `rci` são aterradas de forma fixa em '0', garantindo que ele conte de forma contínua a cada pulso de clock de 1Hz.
- Na linha 89, o componente `timeflags` é instanciado, monitorando constantemente os barramentos de unidade e dezena do contador para ativar os gatilhos temporais correspondentes.
- Na linha 100, a `majestados` é instanciada e conectada ao sistema, recebendo as leituras dos sensores externos e das flags de tempo para comandar diretamente as cores que serão exibidas nos semáforos A e B.

Dessa forma, o sistema completo do controle de semáforos está implementado. Com isso podemos simular o mesmo, para isso foi feito um testbench para simular:


```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity exp8visto3_tb is
5  -- Testbench não possui portas externas
6  end exp8visto3_tb;
7
8  architecture sim of exp8visto3_tb is
9
10     -- Declaração do Componente do Sistema Completo (UUT)
11     component exp8visto3 is
12         port (
13             clock          : in  std_logic;
14             reset           : in  std_logic;
15             ligadesliga     : in  std_logic;
16             sensorA         : in  std_logic;
17             sensorB         : in  std_logic;
18             semaforoA       : out std_logic_vector(2 downto 0);
19             semaforoB       : out std_logic_vector(2 downto 0);
20             contador_uni    : out std_logic_vector(3 downto 0);
21             contador_dez    : out std_logic_vector(3 downto 0)
22         );
23     end component;
24
25     -- Sinais para conectar na UUT
26     signal clk_tb          : std_logic := '0';
27     signal rst_tb          : std_logic := '0';
28     signal ligadesliga_tb  : std_logic := '1'; -- Começa ligado (funcionamento normal)
29     signal sensorA_tb      : std_logic := '0';
30     signal sensorB_tb      : std_logic := '0';
31     signal semaforoA_tb    : std_logic_vector(2 downto 0);
32     signal semaforoB_tb    : std_logic_vector(2 downto 0);
33     signal uni_tb          : std_logic_vector(3 downto 0);
34     signal dez_tb          : std_logic_vector(3 downto 0);
35
36     -- 1 ciclo de clock = 10 ns (representando 1 segundo do mundo real)
37     constant CLK_PERIOD : time := 10 ns;
38
39     begin
40
41         -- Instanciação do Sistema Completo
42         UUT: exp8visto3
43             port map (

```

```

44         clock      => clk_tb,
45         reset      => rst_tb,
46         ligadesliga => ligadesliga_tb,
47         sensorA     => sensorA_tb,
48         sensorB     => sensorB_tb,
49         semaforoA    => semaforoA_tb,
50         semaforoB    => semaforoB_tb,
51         contador_uni => uni_tb,
52         contador_dez => dez_tb
53     );
54
55     -- Gerador de Clock (1Hz simulado em passos de 10ns)
56     clk_process : process
57     begin
58         while true loop
59             clk_tb <= '0';
60             wait for CLK_PERIOD / 2;
61             clk_tb <= '1';
62             wait for CLK_PERIOD / 2;
63         end loop;
64     end process;
65
66     -- Processo de Estímulos (Cenários de Teste)
67     stim_process : process
68     begin
69         -----
70         -- Passo 1: Reset Inicial do Sistema
71         -----
72         rst_tb <= '1';
73         wait for CLK_PERIOD * 2;
74         rst_tb <= '0';
75
76         -- Estado inicial esperado:
77         -- SemaforoA = Verde ("001"), SemaforoB = Vermelho ("100")
78         -- O contador começa a subir a partir de 00.
79
80         -----
81         -- Passo 2: Testar a exceção de corte aos 20s (Carro em B esperando)
82         -----
83         -- Deixa o tempo passar até chegar em 25 segundos (25 ciclos de clock)
84         wait for CLK_PERIOD * 25;

```

```

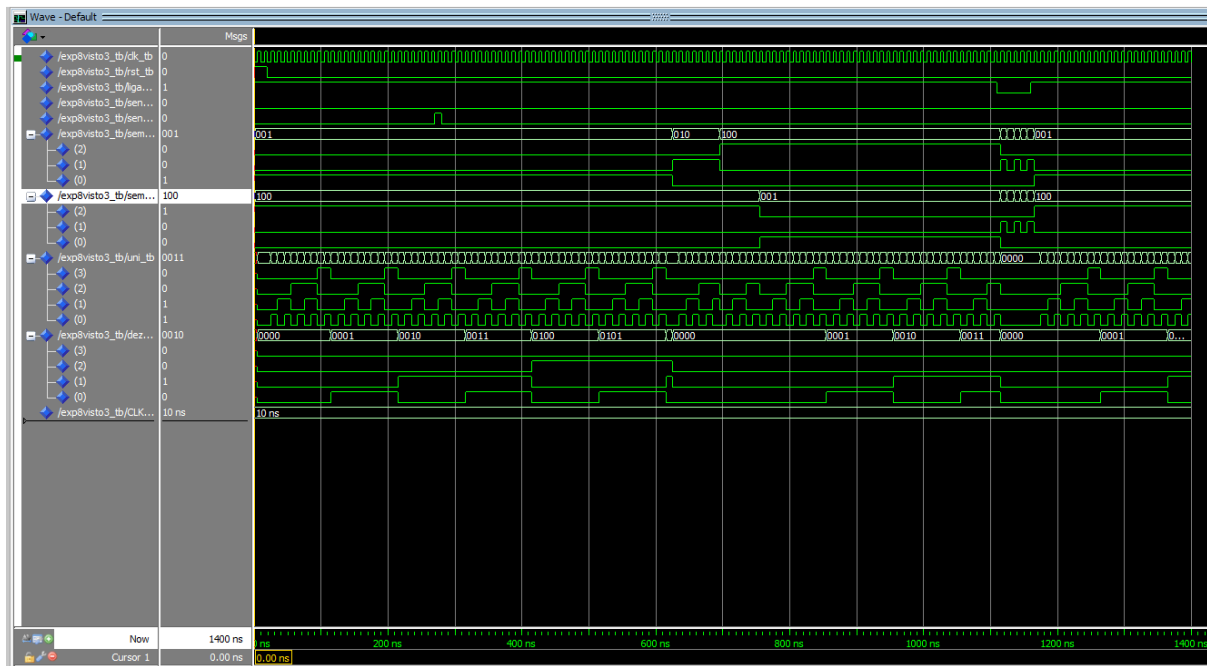
86      -- Passou de 20s. Ativamos o sensorB (carro esperando na pista oposta)
87      sensorB_tb <= '1';
88      sensorA_tb <= '0';
89      wait for CLK_PERIOD;
90
91      -- Como a condição foi atendida, a FSM deve mudar na hora para AMARELO em A ("010")
92      -- e o contador interno deve resetar para "00" para contar os 6s do amarelo.
93      sensorB_tb <= '0'; -- Carro passou, limpa o sensor
94
95      -- Espera os 6s do Amarelo + 5s do Vermelho Ambos (Total: 11 ciclos de clock)
96      wait for CLK_PERIOD * 11;
97
98      -- Agora o SemaforoB abriu (Verde "001") e o SemaforoA fechou (Vermelho "100")
99      -- O contador resetou e começou a contar o tempo da pista B aberta.
100
101      -----
102      -- Passo 3: Testar o tempo limite regulamentar de 60 segundos
103      -----
104      -- Deixamos o semáforo B aberto sem nenhum sensor ativo. Ele deve estourar por tempo.
105      wait for CLK_PERIOD * 61;
106
107      -- Após os 60s, ele vai sozinho para o Amarelo em B (6s) -> Vermelho Ambos (5s) -> Abre A de novo.
108      wait for CLK_PERIOD * 11;
109
110      -----
111      -- Passo 4: Testar o Modo Intermitente (Chave Liga/Desliga)
112      -----
113      ligadesliga_tb <= '0'; -- Desliga o sistema
114
115      -- Espera 5 ciclos para ver o amarelo piscando (Luz "010" -> "000" -> "010" a cada ciclo)
116      wait for CLK_PERIOD * 5;
117
118      ligadesliga_tb <= '1'; -- Religa o sistema (deve voltar ao fluxo normal)
119      wait for CLK_PERIOD * 10;
120
121      -- Finaliza a simulação
122      wait;
123      end process;
124
125      end sim;

```

Listagem 3: Código do testbench.

SIMULAÇÃO E ANÁLISE

Com o testbench obtemos a seguinte onda:



- **0 ns a 20 ns**: Período de **reset** inicial do sistema.
- **20 ns a 270 ns (+250 ns)**: O semáforo Norte/Sul fica verde. Esperamos 25 segundos simulados para testar a aproximação de um carro na outra pista.
- **270 ns a 380 ns (+110 ns)**: O semáforo atende ao sensor: aciona o Amarelo por 6s (60 ns) e depois o Vermelho duplo por 5s (50 ns).
- **380 ns a 990 ns (+610 ns)**: O semáforo Leste/Oeste abre (Verde) e corre pelo tempo regulamentar máximo de 60 segundos (600 ns) até estourar.
- **990 ns a 1100 ns (+110 ns)**: Transição automática de volta: Amarelo na pista oposta por 6s (60 ns) + Vermelho duplo por 5s (50 ns).
- **1100 ns a 1150 ns (+50 ns)**: Ativação da chave **ligadesliga** em zero para testar o modo **Intermitente** (pisca-pisca de emergência) por 5 segundos.
- **1150 ns a 1250 ns (+100 ns)**: O sistema é religado e restabelece o fluxo normal.

COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:

```
# Compile of exp8visto3.vhd was successful.  
# Compile of maqestados.vhd was successful.  
# Compile of tb_questao3.vhd was successful.  
# 3 compiles, 0 failed with no errors.  
  
ModelSim>
```

Figura 15: Compilação dos códigos

Como visto na figura, nenhum código tem algum erro de sintaxe.

CONCLUSÃO

Portanto, concluímos que projetar e implementar o contador BCD 100, a lógica para os semáforos e a máquina de estados para resolvermos o problema dos semáforos proposto pelas questões. Não foram encontrados erros ou divergências neste experimento.